Project Documentation Guidelines

Item	DEPI Industry	DEPI
Project Planning & Management	**2 Oct 2026**	**16 Oct 2026**
Literature Review	**9 Oct 2026**	**16 Oct 2026**
Requirements Gathering	**9 Oct 2026**	**16 Oct 2026**
System Analysis & Design	**9 Oct 2026**	**6 Nov 2026**
Implementation (Source Code & Execution)	**16 Oct 2026**	**30 Nov 2026**
Final Presentation & Testing& Reports	**16 Oct 2026**	**4 Dec 2026**

1. Project Planning & Management

- **Project Proposal: Smart Emergency Response System (SERS)**

- **Overview of the Project:** The Smart Emergency Response System (SERS) is an end-to-end intelligent web application designed to optimize and automate how emergency services and traffic authorities handle road traffic accidents. By leveraging the extensive UK Road Safety Open Dataset (STATS19), the system integrates historical data analytics with a real-time Machine Learning (ML) prediction pipeline. When a new accident is reported, a dispatcher inputs initial scene conditions (e.g., location, weather, lighting, vehicle type) into the system. The underlying ML model instantly predicts collision severity (Fatal, Serious, or Slight). Based on this prediction, the backend system dynamically triggers automated workflows—such as alerting nearby hospitals to prepare emergency beds and prioritizing ambulance dispatch for severe incidents—while simultaneously offering traffic authorities an interactive dashboard to analyze historical accident hotspots.

Project Objectives

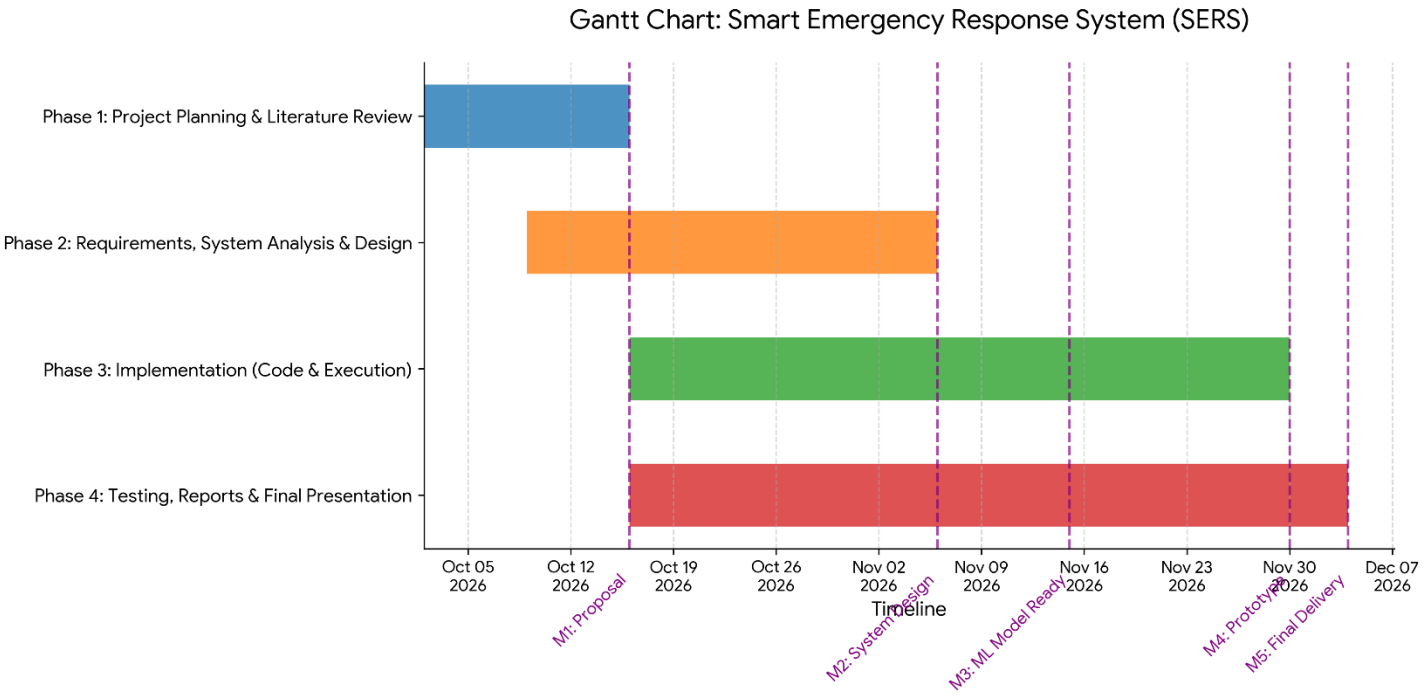
- **Rapid Severity Prediction:** Develop and integrate an accurate Machine Learning classification model capable of instantly predicting accident severity in real-time.
- **Optimized Resource Allocation:** Automate the decision-making process by sending immediate alerts to the nearest hospitals and ambulance units for high-severity accidents, thereby reducing human delays.
- **Data-Driven Traffic Management:** Provide traffic authorities and policymakers with an interactive analytics dashboard featuring historical trend analysis to identify high-risk areas and improve road safety infrastructure.
- **Centralized Role-Based Platform:** Build a secure, multi-page web application customized for different stakeholders (Emergency Dispatchers, Hospital Managers, and Traffic Authorities) to ensure a smooth flow of critical information.

Project Scope

- **In-Scope:**
    - Data cleaning, preprocessing, and exploratory data analysis (EDA) of the UK STATS19 dataset (Collisions, Vehicles, and Casualties).
    - Training and evaluating an ML classification model (e.g., Random Forest, XGBoost) for severity prediction.
    - Developing a robust backend architecture using **FastAPI** to serve the ML model and manage automated routing logic.
    - Designing a relational **Database** to securely log incident reports and predictions.
    - Developing a multi-page **Web Application** containing: a Secure Login Page, an Emergency Dispatcher Input Form, a Live Emergency Feed for hospitals, and an Analytics Dashboard for traffic authorities.
- **Out-of-Scope:**
    - The project will not include the installation or integration of physical IoT sensors or traffic cameras on roads; it relies on manual data entry by dispatchers or simulated API integrations to trigger the workflow.

- **Project Plan**

1. **Timeline & Phases (Gantt Chart Representation)**



- **Phase 1: Project Planning & Literature Review** (Oct 2, 2026 – Oct 16, 2026)
- **Phase 2: Requirements Gathering, System Analysis & Design** (Oct 9, 2026 – Nov 6, 2026)
- **Phase 3: Implementation (Source Code & Execution)** (Oct 16, 2026 – Nov 30, 2026)
- **Phase 4: Testing, Reports & Final Presentation** (Oct 16, 2026 – Dec 4, 2026)

2. **Key Milestones & Deliverables**

- **Milestone 1 (Oct 16):** Submission of the Project Proposal, Gantt chart, and literature review. *(Deliverable: Project Management Document)*

- **Milestone 2 (Nov 6):** Completion of System Architecture (UML diagrams) and Database design (ERD). *(Deliverable: System Analysis & Design Document)*

- **Milestone 3 (Nov 15):** Completion of Data Preprocessing, EDA, and Machine Learning Model training. *(Deliverable: Trained ML Model & Jupyter Notebooks)*

- **Milestone 4 (Nov 30):** Full integration of the FastAPI backend with the Web Application (Frontend) and Database. *(Deliverable: Working Prototype / Source Code on GitHub)*

- **Milestone 5 (Dec 4):** Final system testing, bug fixing, and presentation readiness. *(Deliverable: Final Presentation, Video Demo, and Infographic)*

3. Resource Allocation

Human Resources (Team of 5 Roles):

- **Member 1: Team Leader & System Analyst:** Manages the project timeline, oversees documentation, and designs the high-level system architecture.
- **Member 2: Data Scientist & ML Engineer:** Handles the STATS19 dataset preprocessing, exploratory data analysis (EDA), and the development of general Machine Learning models for severity prediction.
- **Member 3: Backend Developer:** Responsible for building the API endpoints, handling business logic, and database integration and management.
- **Member 4: Frontend Developer:** Focuses on UI/UX design, building the web application interfaces, and integrating interactive analytical dashboards.
- **Member 5: QA & Deployment Engineer:** Executes system testing (Quality Assurance), bug tracking, and manages the cloud deployment and hosting processes.

Technical Resources & Tools (General Tech Stack):

- **Project Management & Collaboration:** Version Control systems (e.g., GitHub/GitLab) and task tracking tools (e.g., Jira, Trello, or Notion).
- **Development Environments:** IDEs like VS Code, and notebook environments like Jupyter Notebooks or Google Colab.
- **Machine Learning & Data Science:** Python, general Machine Learning and Deep Learning frameworks, and standard data manipulation/visualization libraries.
- **Backend & APIs:** Modern Python web frameworks (such as FastAPI or Flask) to serve the models and handle requests.
- **Databases:** Relational (SQL) or Non-Relational (NoSQL) Database Management Systems for securely storing accident records, user data, and system logs.
- **Frontend & Visualizations:** Standard web development technologies (HTML, CSS, JavaScript), frontend frameworks, and interactive dashboarding libraries.
- **Deployment & DevOps:** Containerization tools (e.g., Docker) and Cloud Hosting Platforms (e.g., AWS, Azure, GCP, or Render) to deploy the final system.

- ## Task Assignment & Roles

Team Leader & System Analyst

- **Assigned to:** [Kareem Mohamed Abd El-Nazeer]
- **Responsibilities:**
    - Overseeing the project timeline, managing milestones, and updating the Gantt chart.
    - Gathering functional and non-functional requirements from stakeholders.
    - Designing the overall system architecture, including Use Case, Sequence, and Activity diagrams.
    - Coordinating tasks between team members to ensure smooth technical integration.

2. Data Scientist & Machine Learning Engineer

- **Assigned to:** [Amira Tarek Mahmoud]
- **Responsibilities:**
    - Extracting, cleaning, and preprocessing the UK STATS19 dataset.
    - Conducting Exploratory Data Analysis (EDA) to identify patterns and accident hotspots.
    - Selecting, training, optimizing, and evaluating the Machine Learning classification model for severity prediction.

3. Backend Developer & Database Administrator

- **Assigned to:** [Soha Mohammed Abdel-Aziz]
- **Responsibilities:**
    - Developing the backend infrastructure and robust RESTful APIs (e.g., using FastAPI).
    - Designing the Entity-Relationship Diagram (ERD) and managing the database system.
    - Integrating the trained ML model into the backend.
    - Programming business logic for automated alerts and resource routing based on model predictions.

4. Frontend Developer & UI/UX Designer

- **Assigned to:** [Hagar Seyam Talaat]
- **Responsibilities:**
    - Designing wireframes, mockups, and defining UI/UX guidelines for the web application.
    - Developing interactive user interfaces (Dispatcher Input Form, Live Emergency Feed, and Analytics Dashboard).
    - Ensuring a responsive design and seamless data integration with the backend APIs.

5. Quality Assurance (QA) & Deployment Engineer

- **Assigned to:** [Mohamed Hesham Zaky]
- **Responsibilities:**
    - Writing detailed test plans, executing test cases, and tracking bug reports.
    - Ensuring code quality, proper error handling, and system security.
    - Managing version control (GitHub) branching strategies.
    - Deploying the final web application and ML model to a cloud hosting environment.

- ## Risk Assessment & Mitigation Plan

1. Data-Related Risks

- **Risk:** Highly imbalanced dataset (Fatal and Serious accidents are significantly fewer than Slight accidents), which may cause the ML model to be biased toward predicting "Slight" severity.
  - **Mitigation:** Apply data balancing techniques such as SMOTE (Synthetic Minority Over-sampling Technique), use class weights during model training, and evaluate the model using F1-score and Recall instead of generic Accuracy.

- **Risk:** High volume of missing or unknown values (encoded as -1 or 99 in the STATS19 dataset).
  - **Mitigation:** Perform rigorous data cleaning, use proper imputation methods for missing values, and select robust ML algorithms (like XGBoost or Random Forest) that handle missing data well.

2. Technical & Integration Risks

- **Risk:** Integration issues between the Machine Learning model, the FastAPI backend, and the Frontend application.
  - **Mitigation:** Adopt an API-first approach using FastAPI's built-in Swagger UI documentation. Enforce strict JSON response formats and conduct early integration testing (Postman) before finalizing the UI.

- **Risk:** High prediction latency (system taking too long to return the severity prediction).
  - **Mitigation:** Optimize the ML model size (e.g., using model serialization via joblib or pickle), avoid heavy computations on the fly, and ensure the backend is asynchronous.

3. Project Management & Team Risks

- **Risk:** Missing project deadlines due to scope creep or miscommunication among the 5 team members.
  - **Mitigation:** Strictly follow the Gantt chart, hold brief weekly sync meetings, use a task board (e.g., Trello or Jira), and focus on delivering a Minimum Viable Product (MVP) before adding extra features.

- **Risk:** Version control conflicts and code overwriting on GitHub.
  - **Mitigation:** Implement a strict Git branching strategy (e.g., Feature Branching). No direct commits to the main branch; all changes must pass through Pull Requests (PRs) reviewed by another team member.

- **Key Performance Indicators (KPIs)**

1. **Machine Learning Performance Metrics**

    - **Recall and F1-Score (Target: > 75% for Severe Cases):** Due to the imbalanced nature of accident datasets, Accuracy alone is insufficient. High Recall is crucial to ensure the model rarely misses a "Fatal" or "Serious" accident.
    - **Prediction Latency (Target: < 3 Second):** The time taken from the moment the dispatcher submits the accident data until the ML model returns the severity classification must be instantaneous.

2. **System & Technical Metrics**

    - **API Response Time (Target: < 700 ms):** The FastAPI backend must handle incoming requests and database logging swiftly without bottlenecks.
    - **System Uptime (Target: 99.9%):** Since this is an emergency system, the deployed web application and backend servers must remain highly available during the evaluation and deployment phases.

3. **Business & Usability Metrics**

    - **Automated Alert Success Rate (Target: 100%):** Every accident predicted as "Fatal" or "Serious" must successfully trigger the automated mock alerts (routing to hospitals and ambulances) without failure.
    - **Dashboard Render Time (Target: < 6 Seconds):** The interactive analytics dashboard and heatmaps must load historical STATS19 data quickly to provide a seamless User Experience (UX) for traffic authorities.

2. Literature Review

- **Feedback & Evaluation (Lecturer's Assessment)**

The initial project proposal for the Smart Emergency Response System (SERS) was reviewed and approved by the course lecturer. The core architecture—integrating an analytical dashboard with a FastAPI backend for automated emergency routing—was commended for its practical business value and end-to-end workflow. However, the lecturer provided a critical technical assessment regarding the predictive modeling phase: Given the massive scale of the combined UK STATS19 datasets (hundreds of thousands of records across Collisions, Vehicles, and Casualties), relying solely on traditional Machine Learning algorithms (e.g., Random Forest or XGBoost) may limit performance. The lecturer explicitly instructed the team to utilize Deep Learning (Neural Networks) to better capture complex, non-linear relationships within the large dataset and achieve higher classification accuracy.

- **Suggested Improvements** – Based on the lecturer's feedback, the project workflow has been updated with the following improvements:
    - **Model Upgrade:** Transition from traditional ML classifiers to a Deep Learning architecture (e.g., Multi-Layer Perceptron - MLP) using frameworks like **TensorFlow/Keras** or **PyTorch**.
    - **Data Pipeline Optimization:** Enhance the data preprocessing pipeline to handle large-scale data efficiently, utilizing data batching and advanced scaling techniques suitable for Neural Networks.
    - **Latency Management:** Since Neural Networks can be computationally heavy, ensure the trained model is highly optimized (e.g., using model quantization or ONNX runtime) to maintain the strict KPI of < 1 Second prediction latency during FastAPI deployment.

- **Final Grading Criteria** – Breakdown of marks based on documentation, implementation, testing, and presentation.

3. Requirements Gathering

- ## Stakeholder Analysis

 Emergency Dispatchers (Call Center Agents): Need a fast, intuitive, and distraction-free interface to input initial accident data while on the phone with callers.
 Hospital Emergency Managers: Need advance, automated warnings of incoming severe casualties to proactively prepare trauma rooms and medical staff.
 Ambulance Services / Paramedics: Need precise GPS coordinates and immediate dispatch prioritization for high-severity crashes.
 Traffic Authorities (Police & Department for Transport): Need analytical dashboards with interactive maps and historical data to identify road hazards and plan infrastructure improvements.

- ## User Stories & Use Cases

 As *an Emergency Dispatcher or normal user,* I want to quickly input weather, location, and vehicle data into the system, so that I can instantly know the predicted severity of the accident.
 As a Hospital Manager, I want to receive automated alerts for "Fatal" or "Serious" accidents near my facility, so that my team can prepare beds before the ambulance arrives.
 As a Traffic Analyst, I want to view a heatmap of historical accident hotspots, so that I can recommend installing better lighting or speed cameras in dangerous areas.

Key Use Cases:

- **UC-01 (Submit Accident Report):** User logs in, fills out the emergency form, and submits it to the FastAPI backend.
- **UC-02 (Predict Severity):** The backend passes the data to the ML OR DL (Neural Network) model, which returns the classification (Fatal, Serious, Slight).
- **UC-03 (Automated Routing & Alerts):** The system checks the prediction; if Severe/Fatal, it queries the database for the nearest hospital and triggers an automated Webhook/API alert.
- **UC-04 (View Analytics Dashboard):** Authorized users access the dashboard to filter and visualize historical STATS19 data using interactive charts.

- ## Functional Requirements

 Authentication: The system must provide a secure login page with Role-Based Access Control (RBAC) to differentiate between dispatchers, hospital staff, and traffic analysts.
 Data Input Form: The system must allow users to input critical accident parameters (e.g., coordinates, weather conditions, lighting, road type, vehicle type).
 Real-Time ML Prediction: The system must utilize a pre-trained Neural Network model to classify accident severity instantly upon form submission.
 Automated Alerting Logic: The system must automatically trigger simulated alerts (mock emails, SMS, or webhooks) to designated hospitals when an accident is predicted as "Serious" or "Fatal".
 Database Logging: The system must securely save every reported incident, its input parameters, and the model's prediction into the database (SQL/NoSQL).
 Interactive Dashboard: The system must render visual analytics (heatmaps, bar charts, time-series graphs) based on the historical STATS19 dataset.

- **Non-functional Requirements**

 Performance & Latency: The Deep Learning model inference and the FastAPI response must take less than 1 second to ensure rapid emergency handling.
 Security: User authentication must be implemented using JWT (JSON Web Tokens). Passwords must be hashed, and data transmission must be secured via HTTPS.
 Reliability & Availability: As an emergency platform, the system targets a 99.9% uptime.
 Usability: The frontend UI must be responsive, ensuring it is fully operational on both desktop monitors in dispatch centers and mobile tablets in ambulances.
 Scalability: The FastAPI backend must be implemented asynchronously (async def) to handle multiple concurrent emergency reports without blocking operations.

4. System Analysis & Design

1.Problem Statement & Objectives.

Problem: In the event of a traffic accident, manual assessment of the severity of the accident and dispatching emergency services usually suffer from critical time delays. Relying solely on human intuition to evaluate conditions (like the weather or road type) can lead to inefficient allocation of limited resources, such as ICU beds and ambulances that move based on knowledge of the accident and its severity.

Project Objectives:
- To engineer a Smart Emergency Response System (SERS) that replaces manual severity estimation with an automated, data-driven ML or Deep Learning (Neural Network) model.
- To minimize emergency response times by automatically routing high-severity alerts to the medical facilities.
- To provide traffic management stakeholders with an interactive, real-time analytics dashboard based on the UK STATS19 dataset to identify hazard zones.

• Use Case Diagram & Descriptions.

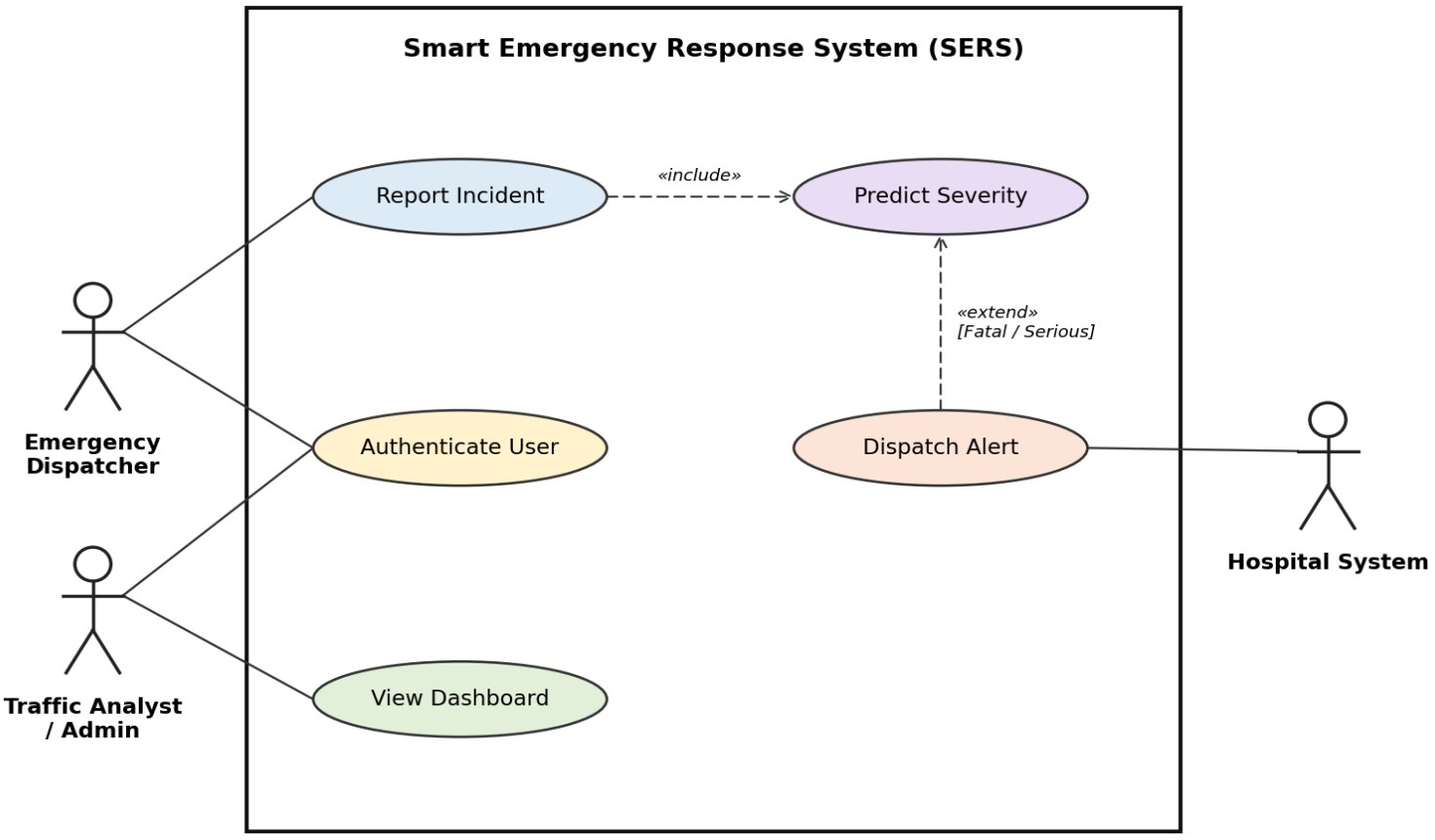
Actors and Goals List

Actor	Goals list
Emergency Dispatcher	• Authenticate User • Report Incident
Traffic Analyst / Admin	• Authenticate User • View Dashboard
Hospital System (secondary)	• Dispatch Alert

Use Case Descriptions

ID	Use Case	Actor	Relationship	Description
UC-01	Authenticate User	Emergency Dispatcher, Traffic Analyst / Admin	Association	Users log in securely to the web application. The system assigns permissions based on their role (RBAC).
UC-02	Report Incident	Emergency Dispatcher	Association	The dispatcher submits a form containing incident variables (coordinates, weather, light conditions, vehicle type).
UC-03	Predict Severity	System	«include» in UC-02	The FastAPI backend passes the submitted data to the Neural Network, which instantly predicts the severity (Fatal, Serious, Slight).
UC-04	Dispatch Alert	System, Hospital System	«extend» UC-03 [Fatal / Serious]	If the predicted severity is Fatal or Serious, the system finds the nearest hospital and sends an automated alert (Email/Webhook) with the exact GPS coordinates.
UC-05	View Dashboard	Traffic Analyst / Admin	Association	The user views accident patterns, severity distributions, and heatmaps based on historical STATS19 data.

Use Case Diagram – SERS



Functional Requirements:

1. The system shall process incident form data and return a severity prediction.
2. The system shall log all incident inputs and model predictions into the relational/NoSQL database.
3. The system shall generate automated alerts for severe incidents using a predefined routing algorithm.
4. The system should render interactive maps and charts for historical data analysis.

Non-Functional Requirements:

1. *Performance:* The prediction inference time must not exceed 1 second.
2. *Architecture:* The backend must be designed using an asynchronous RESTful API architecture.
3. *Security:* All API endpoints must be secured, and sensitive user data (passwords) must be hashed.
4. *Scalability:* The system components must be modularly designed to allow future integration with real IoT sensors or external GPS services in the future.

Software Architecture

Architecture Style

- SERS follows a **3-Tier Layered (Client–Server) architecture** implemented as a **modular monolith** that exposes a **RESTful API**. The frontend, backend, and database are separated into independent tiers, and the backend is organized internally into layers (Controllers → Services → Data Access).

Why this style	Reason
Separation of concerns	The UI team and backend team can work in parallel (5-member team).
Simple deployment	One backend service and one database are enough for the project scope and timeline.
Low latency	The Deep Learning model runs inside the FastAPI process, so there is no network hop to a separate ML service (target: < 1 second).
Scalability path	Services are modular, so they can later be split into microservices or connected to real IoT/GPS sources.

- **Why not Microservices?** They add network latency, orchestration overhead, and DevOps complexity that are not justified for a 5-person team with a ~2-month schedule.

MVC Mapping

MVC	In SERS
View	Web application (Login, Incident Input Form, Live Emergency Feed, Analytics Dashboard)
Controller	FastAPI routers (/auth, /incidents, /alerts, /analytics)
Model	ORM models, services, and the Deep Learning prediction model

Tier	Component	Responsibility
Presentation	Login Page	Secure authentication for all roles.
	Incident Input Form	Dispatcher enters coordinates, weather, light conditions, road type, vehicle type.
	Live Emergency Feed	Hospital staff see incoming Fatal/Serious alerts.
	Analytics Dashboard	Heatmaps, bar charts, and time-series from STATS19 data.
Application	API Layer (Controllers)	Receives requests, validates input, returns JSON responses.
	Security Middleware	JWT authentication, Role-Based Access Control (RBAC), password hashing.
	Incident Service	Orchestrates the flow: saves the incident, calls prediction, then triggers alerts.
	Prediction Service	Preprocesses inputs and runs the Neural Network; returns Fatal / Serious / Slight.
	Alert & Routing Service	Finds the nearest hospital and sends a mock alert (Webhook/Email/SMS) for Fatal/Serious cases.
	Analytics Service	Aggregates historical data for charts and heatmaps.
	Data Access Layer	ORM models and repositories that isolate the database from the business logic.
Data	Relational Database	Users, incidents, predictions, hospitals, alerts, logs. System Components
	Historical STATS19 Store	Cleaned/aggregated dataset used by the dashboard.
	Model Artifacts	Trained model + scaler, loaded once at backend startup.
External	Hospital System (mock)	Receives automated alerts.
	ML Training Pipeline (offline)	Jupyter/TensorFlow notebooks that train and export the model.

Component Interactions (Main Flow)

1. The dispatcher logs in; the backend verifies credentials and returns a **JWT** token with the user's role.
2. The dispatcher submits the incident form → POST /incidents over HTTPS.
3. The security middleware validates the token and role (RBAC).
4. **Incident Service** saves the raw incident in the database.
5. **Prediction Service** preprocesses the data and runs the Neural Network, returning the severity.
6. The prediction is saved and linked to the incident.
7. If severity is **Fatal or Serious**, **Alert & Routing Service** selects the nearest hospital and sends the mock alert with GPS coordinates; the alert is logged.
8. The response (severity + alert status) returns to the dispatcher's screen, and the hospital's Live Emergency Feed shows the new alert.
9. Traffic analysts open the dashboard → GET /analytics, and **Analytics Service** returns aggregated STATS19 data.

Architecture vs. Non-Functional Requirements

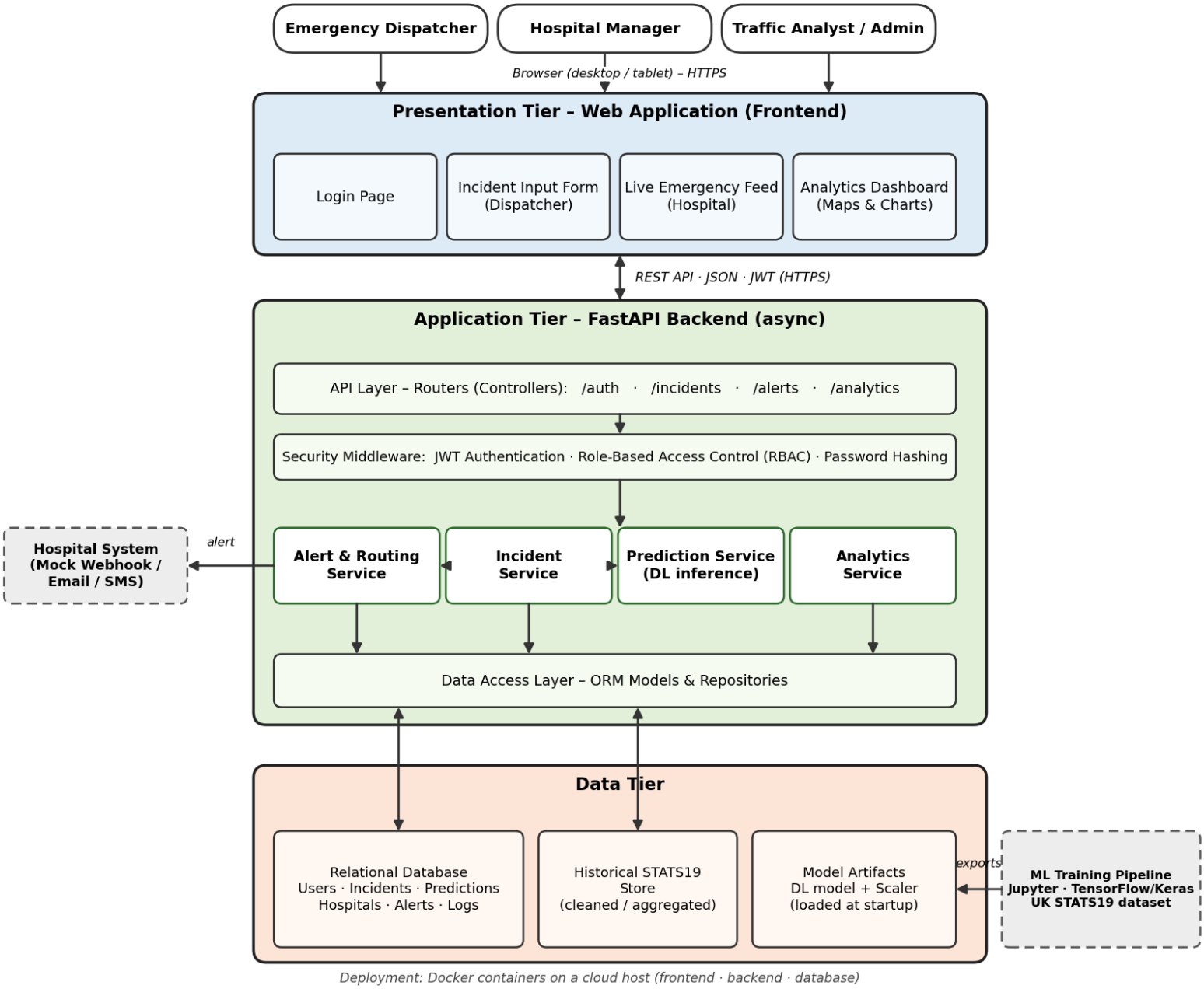
NFR	How the architecture supports it
Performance (< 1 s)	The model is loaded at startup and runs in-process; optimized with ONNX/quantization.
Scalability / Concurrency	Asynchronous FastAPI (`async def`) and stateless API, so more instances can run behind a load balancer.
Security	JWT, RBAC, hashed passwords, HTTPS, input validation in the API layer.
Availability (99.9%)	Docker containers on a cloud host, with a health-check endpoint.
Usability	Responsive web UI for dispatch-center desktops and ambulance tablets.
Modularity	Independent services, so the mock alert can be replaced by a real hospital API without changing other layers.

Technology Stack

Layer	Technology
Frontend	HTML/CSS/JavaScript with a frontend framework, plus a mapping/charting library for heatmaps
Backend	Python, FastAPI (async), Pydantic
Machine Learning	TensorFlow/Keras or PyTorch (MLP), exported to ONNX
Database	PostgreSQL (relational)
Security	JWT, bcrypt, HTTPS/TLS
DevOps	Docker, GitHub, cloud hosting (e.g., Render, AWS, Azure)

Software Architecture – Smart Emergency Response System (SERS)

Style: 3-Tier Layered (Client–Server) · Modular Monolith · RESTful API



Deployment: Docker containers on a cloud host (frontend · backend · database)

Database Design & Data Modeling

UK Road Casualty Statistics 2025 (STATS19) – DEPI Project

1. ER Diagram

Figure 1 – ER diagram (crow's foot notation, created with dbdiagram.io). *Insert the exported ERD image here.*

1.1 Entities

- **Collision** – a *strong entity*. Identified by `collision_index`.
- **Vehicle** – a *weak (identification-dependent) entity*. A vehicle number only has meaning inside one collision, so its key is `(collision_index, vehicle_reference)`.
- **Casualty** – a *weak (identification-dependent) entity*. A casualty number only has meaning inside one collision, so its key is `(collision_index, casualty_reference)`.

1.2 Relationships

Relationship	Parent	Child	Cardinality	Foreign key
A collision involves vehicles	collision	vehicle	1 : N (at least 1 vehicle)	vehicle.collision_index
A collision causes casualties	collision	casualty	1 : N (at least 1 casualty)	casualty.collision_index
A casualty is linked to a vehicle	vehicle	casualty	1 : N (a vehicle may have 0..N casualties; each casualty belongs to exactly 1 vehicle)	casualty.(collision_index, vehicle_reference)

2. Logical Schema

The logical schema is DBMS-independent. Logical types: **Identifier**, **Category** (decoded text label from a coded list), **Integer**, **Decimal**, **Date**, **Time**.

Keys.

- COLLISION (**collision_index**)
- VEHICLE (**collision_index**, **vehicle_reference**), FK (collision_index) → COLLISION
- CASUALTY (**collision_index**, **casualty_reference**), FK (collision_index) → COLLISION, FK (collision_index, vehicle_reference) → VEHICLE

2.1 COLLISION

Attribute	Logical type	Constraint	Description
collision_index	Identifier	PK, NOT NULL	Unique collision ID (13 characters)
longitude	Decimal		Longitude of the collision
latitude	Decimal		Latitude of the collision
police_force	Category		Reporting police force (44 values)
collision_severity	Category	Fatal / Serious / Slight	Severity of the collision
number_of_vehicles	Integer	≥ 1	Vehicles involved
number_of_casualties	Integer	≥ 1	Casualties in the collision
date	Date		Collision date (2025)
day_of_week	Category		Day name
time	Time		Time of day (HH:MM)
local_authority_ons_district	Category		Local authority district (351 values)
local_authority_highway	Category		Highway authority
local_authority_highway_current	Category		Current highway authority
first_road_class	Category		Class of the first road
first_road_number	Integer		Number of the first road
road_type	Category		Road layout (carriageway type)

Attribute	Logical type	Constraint	Description
speed_limit	Integer	20–70 mph	Posted speed limit
junction_detail	Category		Junction type
junction_control	Category		Junction control
second_road_class	Category		Class of the second road
second_road_number	Integer		Number of the second road
pedestrian_crossing	Category		Pedestrian crossing facility near the location
light_conditions	Category		Lighting conditions
weather_conditions	Category		Weather conditions
road_surface_conditions	Category		Road surface state
special_conditions_at_site	Category		Special conditions at the site
carriageway_hazards	Category		Hazards on the carriageway
urban_or_rural_area	Category		Urban or Rural
did_police_officer_attend_scene_of_accident	Category		Police attendance
trunk_road_flag	Category		Trunk road indicator
lsoa_of_accident_location	Identifier		LSOA code of the location
enhanced_severity_collision	Category		DfT enhanced severity classification
collision_injury_based	Category		Injury-based reporting indicator
collision_adjusted_severity_serious	Decimal		DfT adjusted-severity estimate (serious)
collision_adjusted_severity_slight	Decimal		DfT adjusted-severity estimate (slight)

2.2 VEHICLE

Attribute	Logical type	Constraint	Description
collision_index	Identifier	PK, FK → collision	Parent collision
vehicle_reference	Integer	PK, NOT NULL	Vehicle number within the collision
vehicle_type	Category		Type of vehicle (22 values)
towing_and_articulation	Category		Towing / articulation
vehicle_manoeuvre	Category		Manoeuvre before the collision
vehicle_direction_from	Category		Direction travelled from
vehicle_direction_to	Category		Direction travelled to
vehicle_location_restricted_lane	Category		Restricted lane location
junction_location	Category		Position relative to the junction
skidding_and_overturning	Category		Skidding / overturning
hit_object_in_carriageway	Category		Object hit in the carriageway
vehicle_leaving_carriageway	Category		Left the carriageway
hit_object_off_carriageway	Category		Object hit off the carriageway
first_point_of_impact	Category		First point of impact
vehicle_left_hand_drive	Category		Left-hand drive vehicle
journey_purpose_of_driver	Category		Purpose of the journey
sex_of_driver	Category		Driver sex

Attribute	Logical type	Constraint	Description
age_of_driver	Integer	0–120	Driver age
age_band_of_driver	Category		Driver age band (derived from age)
engine_capacity_cc	Integer	≥ 0	Engine size in cc
propulsion_code	Category		Propulsion / fuel type
generic_make_model	Category		Generic make and model
driver_imd_decile	Category		Deprivation decile of the driver's area
lsoa_of_driver	Identifier		LSOA code of the driver
escooter_flag	Category		E-scooter indicator
driver_distance_banding	Category		Distance from home (band)

2.3 CASUALTY

Attribute	Logical type	Constraint	Description
collision_index	Identifier	PK, FK → collision	Parent collision
vehicle_reference	Integer	FK → vehicle, NOT NULL	Linked vehicle (part of the composite FK)
casualty_reference	Integer	PK, NOT NULL	Casualty number within the collision
casualty_class	Category		Driver or rider / Passenger / Pedestrian
sex_of_casualty	Category		Casualty sex
age_of_casualty	Integer	0–120	Casualty age
age_band_of_casualty	Category		Age band (derived from age)
casualty_severity	Category	Fatal / Serious / Slight	Severity of the injury
pedestrian_location	Category		Pedestrian location
pedestrian_movement	Category		Pedestrian movement
car_passenger	Category		Car passenger seat position
bus_or_coach_passenger	Category		Bus / coach passenger status
pedestrian_road_maintenance_worker	Category		Road maintenance worker flag
casualty_type	Category		Casualty type (23 values)
casualty_imd_decile	Category		Deprivation decile of the casualty's area
lsoa_of_casualty	Identifier		LSOA code of the casualty
enhanced_casualty_severity	Category		DfT enhanced severity classification
casualty_injury_based	Category		Injury-based reporting indicator
casualty_adjusted_severity_serious	Decimal		DfT adjusted-severity estimate (serious)
casualty_adjusted_severity_slight	Decimal		DfT adjusted-severity estimate (slight)
casualty_distance_banding	Category		Distance from home (band)
escooter_flag	Category		E-scooter indicator

3. Physical Schema

DBMS: SQLite 3 (file `road_safety.db`) with `PRAGMA foreign_keys = ON` so that foreign keys are enforced.

3.1 Type mapping

Logical type	Physical type	Note
Identifier	VARCHAR(13) (collision_index),	Fixed-format codes

Logical type	Physical type	Note
	VARCHAR(20) (LSOA)	
Category	VARCHAR(20) … VARCHAR(150)	Length sized above the longest observed label
Integer	INT	NULL allowed except keys
Decimal	FLOAT	Coordinates and adjusted-severity values
Date / Time	DATE / TIME	Stored as ISO text in SQLite

3.2 Primary and foreign keys

Table	Primary key	Foreign keys
collision	collision_index	–
vehicle	(collision_index, vehicle_reference)	collision_index → collision
casualty	(collision_index, casualty_reference)	collision_index → collision; (collision_index, vehicle_reference) → vehicle

The primary key of *casualty* has two columns because `casualty_reference` is already unique within a collision (0 duplicates). `vehicle_reference` is a NOT NULL foreign key, not part of the primary key.

3.3 DDL

```
CREATE TABLE collision (
    collision_index VARCHAR(13) NOT NULL,
    longitude FLOAT,
    latitude FLOAT,
    police_force VARCHAR(50),
    collision_severity VARCHAR(20),
    number_of_vehicles INT,
    number_of_casualties INT,
    date DATE,
    day_of_week VARCHAR(20),
    time TIME,
    local_authority_ons_district VARCHAR(20),
    local_authority_highway VARCHAR(20),
    local_authority_highway_current VARCHAR(20),
    first_road_class VARCHAR(50),
    first_road_number INT,
    road_type VARCHAR(50),
    speed_limit INT,
    junction_detail VARCHAR(100),
    junction_control VARCHAR(50),
    second_road_class VARCHAR(100),
    second_road_number INT,
    pedestrian_crossing VARCHAR(150),
    light_conditions VARCHAR(50),
    weather_conditions VARCHAR(50),
    road_surface_conditions VARCHAR(50),
    special_conditions_at_site VARCHAR(100),
    carriageway_hazards VARCHAR(150),
    urban_or_rural_area VARCHAR(50),
    did_police_officer_attend_scene_of_accident VARCHAR(150),
    trunk_road_flag VARCHAR(100),
    lsoa_of_accident_location VARCHAR(20),
    enhanced_severity_collision VARCHAR(50),
    collision_injury_based VARCHAR(50),
    collision_adjusted_severity_serious FLOAT,
    collision_adjusted_severity_slight FLOAT,
    PRIMARY KEY (collision_index)
);

CREATE TABLE vehicle (
    collision_index VARCHAR(13) NOT NULL,
    vehicle_reference INT NOT NULL,
    vehicle_type VARCHAR(100),
    towing_and_articulation VARCHAR(50),
```

```
    vehicle_manoeuvre VARCHAR(100),
    vehicle_direction_from VARCHAR(20),
    vehicle_direction_to VARCHAR(20),
    vehicle_location_restricted_lane VARCHAR(150),
    junction_location VARCHAR(100),
    skidding_and_overturning VARCHAR(100),
    hit_object_in_carriageway VARCHAR(100),
    vehicle_leaving_carriageway VARCHAR(100),
    hit_object_off_carriageway VARCHAR(50),
    first_point_of_impact VARCHAR(50),
    vehicle_left_hand_drive VARCHAR(20),
    journey_purpose_of_driver VARCHAR(100),
    sex_of_driver VARCHAR(20),
    age_of_driver INT,
    age_band_of_driver VARCHAR(20),
    engine_capacity_cc INT,
    propulsion_code VARCHAR(50),
    generic_make_model VARCHAR(50),
    driver_imd_decile VARCHAR(50),
    lsoa_of_driver VARCHAR(20),
    escooter_flag VARCHAR(50),
    driver_distance_banding VARCHAR(150),
    PRIMARY KEY (collision_index, vehicle_reference),
    FOREIGN KEY (collision_index) REFERENCES collision (collision_index)
);

CREATE TABLE casualty (
    collision_index VARCHAR(13) NOT NULL,
    vehicle_reference INT NOT NULL,
    casualty_reference INT NOT NULL,
    casualty_class VARCHAR(50),
    sex_of_casualty VARCHAR(20),
    age_of_casualty INT,
    age_band_of_casualty VARCHAR(20),
    casualty_severity VARCHAR(20),
    pedestrian_location VARCHAR(150),
    pedestrian_movement VARCHAR(150),
    car_passenger VARCHAR(50),
    bus_or_coach_passenger VARCHAR(50),
    pedestrian_road_maintenance_worker VARCHAR(50),
    casualty_type VARCHAR(150),
    casualty_imd_decile VARCHAR(50),
    lsoa_of_casualty VARCHAR(20),
    enhanced_casualty_severity VARCHAR(50),
    casualty_injury_based VARCHAR(50),
    casualty_adjusted_severity_serious FLOAT,
    casualty_adjusted_severity_slight FLOAT,
    casualty_distance_banding VARCHAR(150),
    escooter_flag VARCHAR(100),
    PRIMARY KEY (collision_index, casualty_reference),
    FOREIGN KEY (collision_index) REFERENCES collision (collision_index),
    FOREIGN KEY (collision_index, vehicle_reference)
        REFERENCES vehicle (collision_index, vehicle_reference)
);
```

4. Normalization Considerations

4.1 First normal form (1NF) – satisfied

Every attribute holds a single atomic value, there are no repeating groups, and every row is uniquely identified by its primary key. Multi-valued facts (several vehicles or casualties per collision) are stored as separate rows in child tables rather than as repeated columns.

4.2 Second normal form (2NF) – satisfied

Collision has a single-column key, so partial dependencies are impossible. In *vehicle* and *casualty* (composite keys), every non-key attribute describes the specific vehicle or person inside a specific collision, so it depends on the whole key.

4.3 Third normal form (3NF) – partially satisfied

Checking the loaded data for non-key attributes that determine other non-key attributes (or are derivable from other data) found:

Table	Dependency	Evidence in the data	Type
collision	number_of_vehicles = count of vehicle rows	100% match	Derived attribute
collision	number_of_casualties = count of casualty rows	100% match	Derived attribute
collision	collision_severity = worst casualty_severity	100% match	Derived attribute
collision	local_authority_ons_district → local_authority_highway	0 exceptions in 351 districts	Transitive
collision	local_authority_ons_district → police_force	14 exceptions in 351 districts	Near-dependency
collision	lsoa_of_accident_location → local_authority_ons_district	213 exceptions in 27,178 LSOAs	Near-dependency
vehicle	age_of_driver → age_band_of_driver	0 exceptions in 99 ages	Transitive / derived
vehicle	lsoa_of_driver → driver_imd_decile	73 exceptions in 33,457 LSOAs	Near-dependency
casualty	age_of_casualty → age_band_of_casualty	0 exceptions in 102 ages	Transitive / derived
casualty	lsoa_of_casualty → casualty_imd_decile	56 exceptions in 31,358 LSOAs	Near-dependency

4.4 Design decision: controlled denormalization

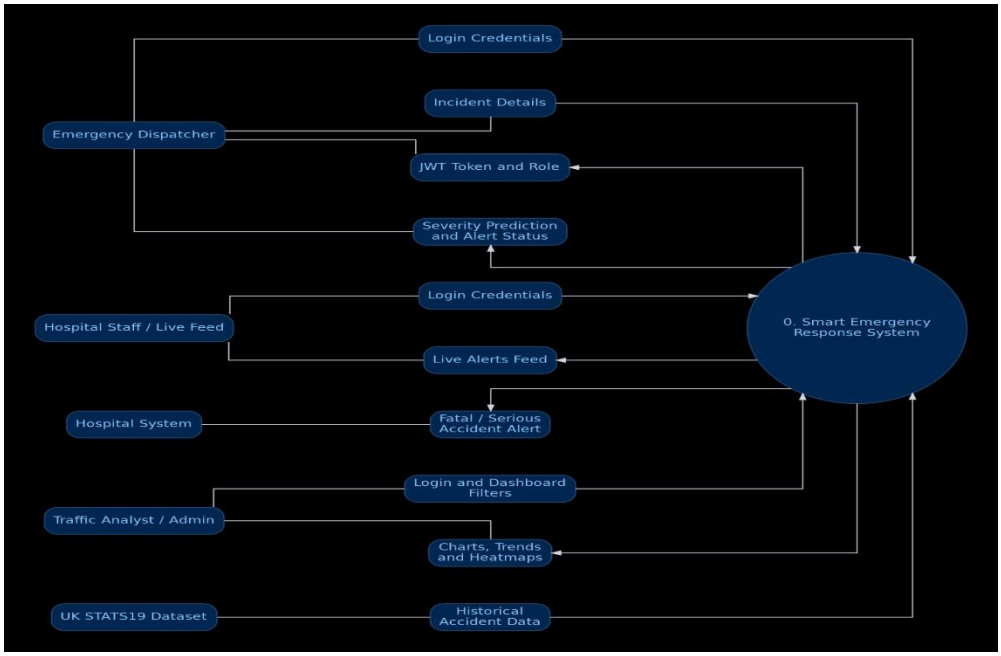
The database keeps three tables and accepts the redundancy above, for these reasons:

1. **The data is published and read-only.** It is an annual release that is loaded once, so update, insert and delete anomalies cannot occur.
2. **The workload is analytical** (aggregations, SQL queries, dashboards). Fewer joins make queries simpler and faster.
3. **Derived columns are used directly** by queries (for example, average casualties per collision uses `number_of_casualties`).
4. **Decoded text labels** keep the data readable without lookup joins.

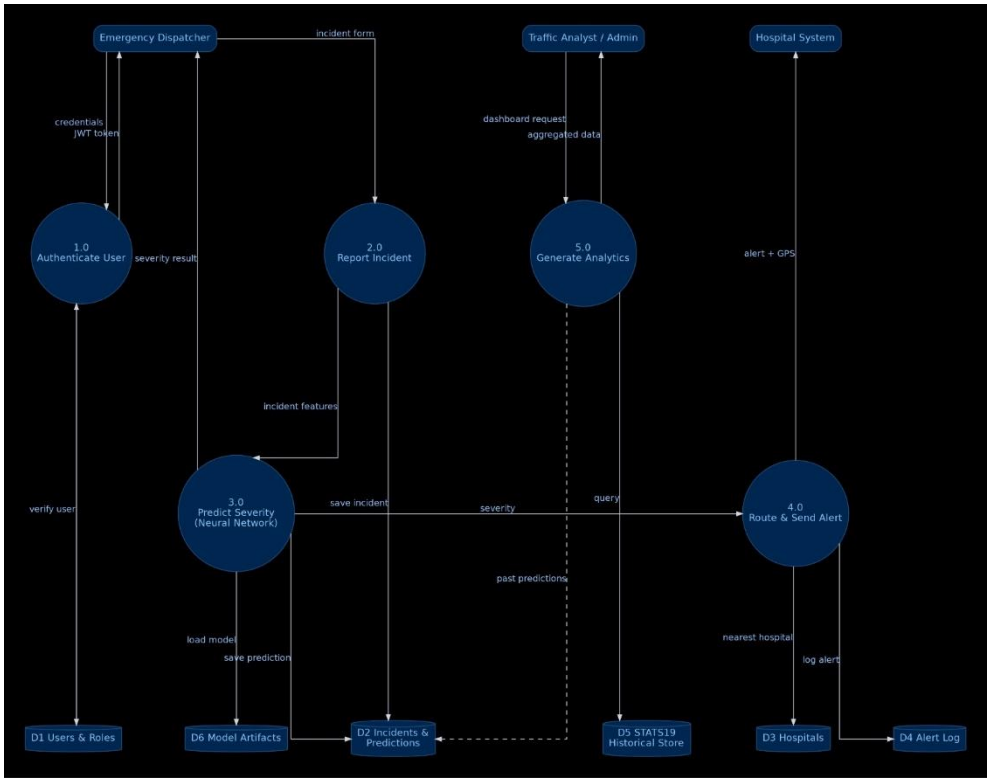
To reach full 3NF, the LSOA, local authority and age-band attributes would move into their own lookup tables, and the derived counts would become views.

3. Data Flow & System Behavior

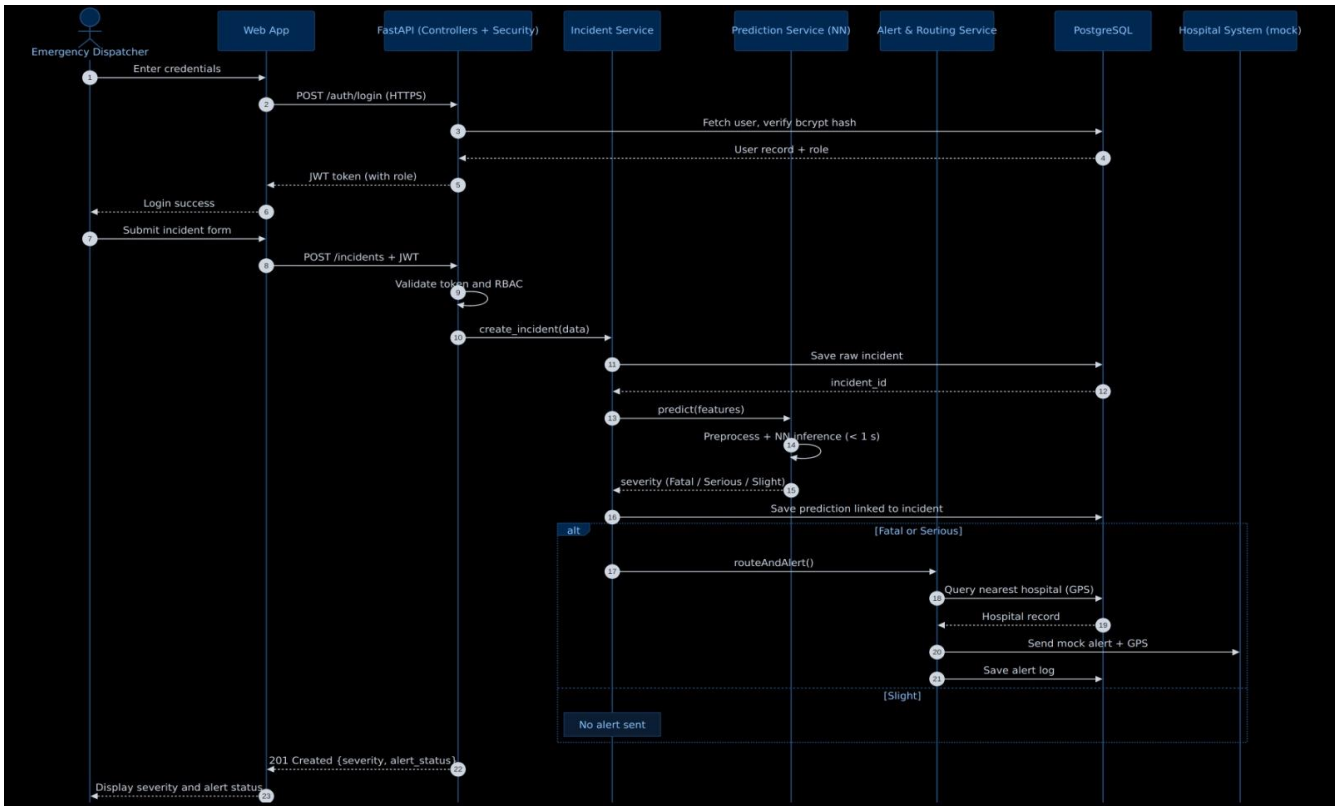
1 DFD context level



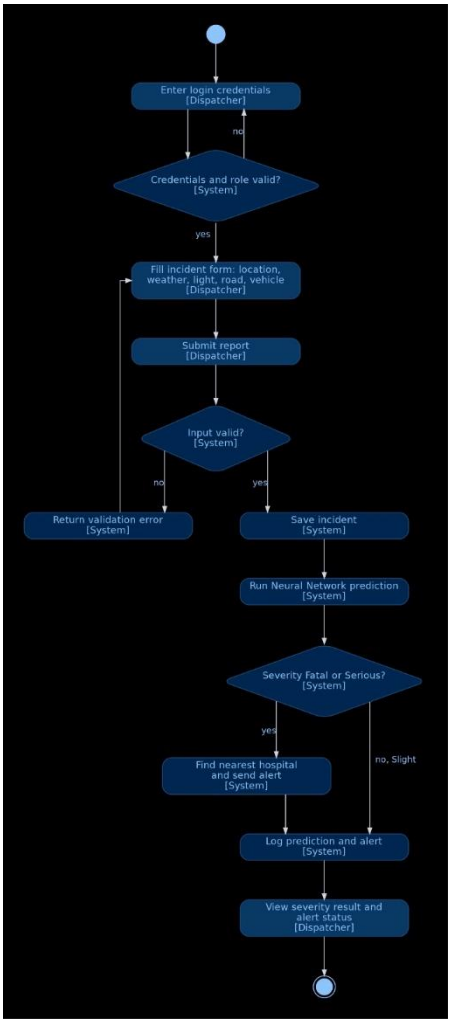
2 DFD level



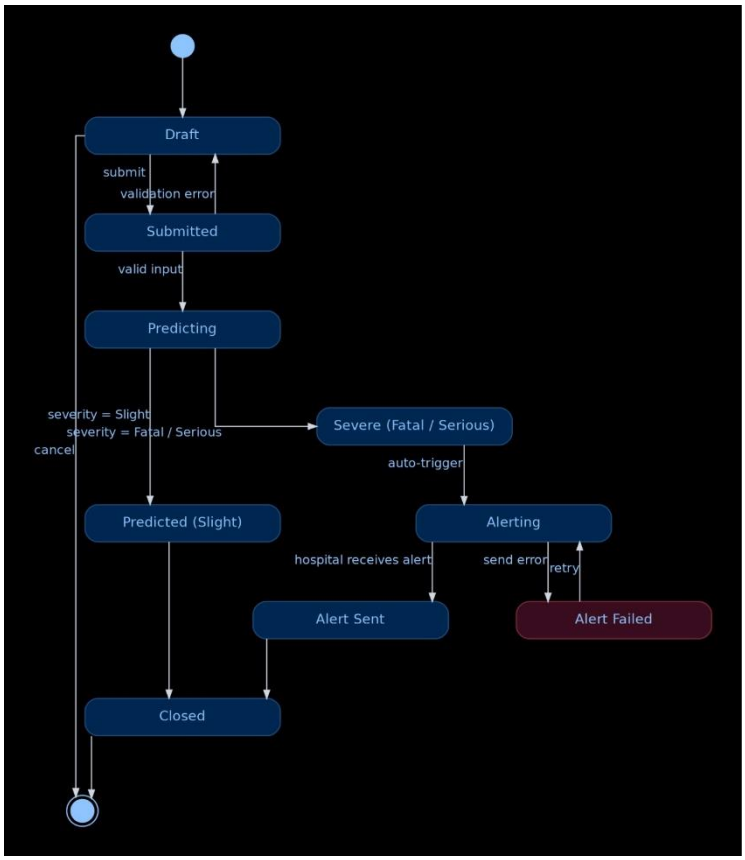
Sequence Diagrams



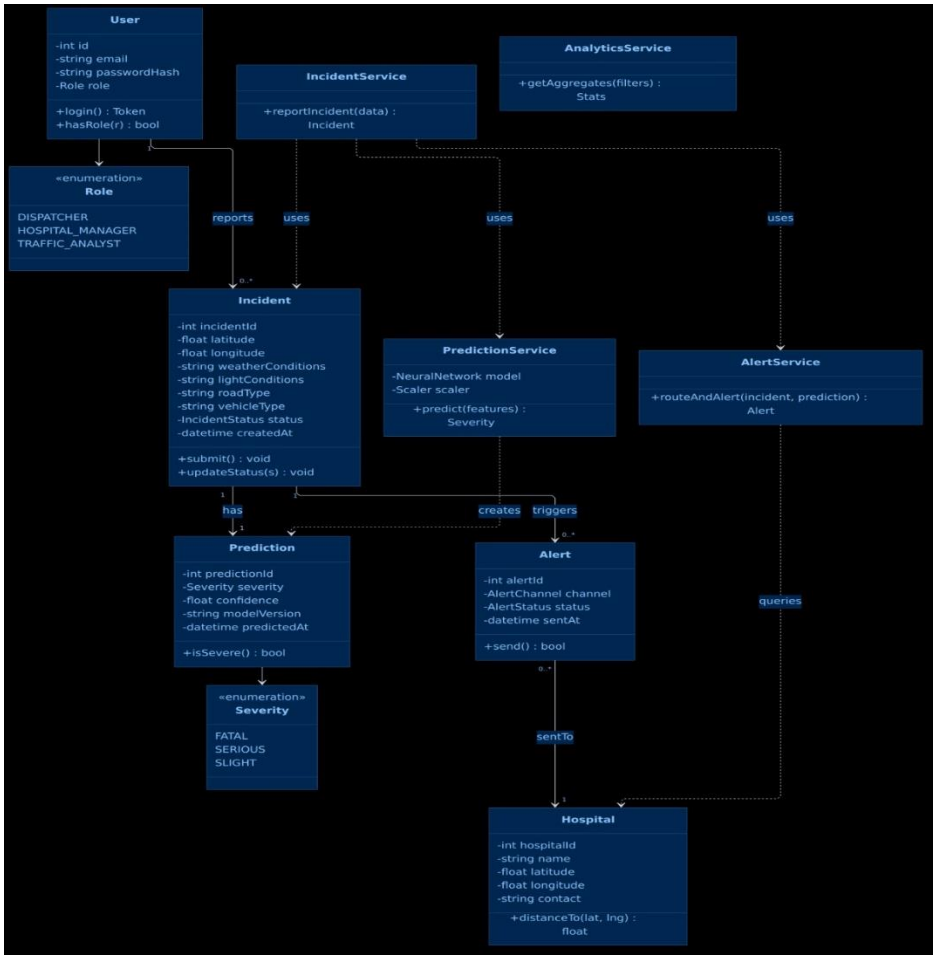
Activity Diagram



State Diagram



Class Diagram



4. UI/UX Design & Prototyping

4.1. UI/UX Guidelines

The user interface for the Smart Emergency Response System (SERS) was meticulously designed to reduce cognitive load for emergency workers operating in high-stress environments. The design system follows these core guidelines:

- **Design Principles (Stress-Aware Design):** The interface adopts a "data-centric and distraction-free" philosophy. Information hierarchy prioritizes critical AI predictions and actionable alerts, utilizing a modular card-based layout to separate inputs from outputs.
- **Color Scheme (High-Contrast & Semantic):**
    - *Primary & Navigation:* Deep Navy Blue (#0B1B3D) is used for the main sidebars and branding to provide a professional, authoritative anchor.
    - *Workspaces:* Crisp White (#FFFFFF) backgrounds with light gray borders for data-entry areas to maximize readability.
    - *Semantic Alerts:* Deep Red (#9B1C1C) is strictly reserved for "Fatal/Serious" AI assessments to immediately draw the eye. Success Green is used for system operational status and "Alert Acknowledged" actions.
- **Typography:** A highly legible, modern sans-serif typeface is utilized across all screens. Large, bold font weights are applied to critical metrics (e.g., Confidence Percentages, Total Collisions) to allow for rapid scanning by dispatchers.
- **Accessibility Considerations:** The design ensures WCAG-compliant contrast ratios between text and backgrounds. Icons are paired with text labels (e.g., weather icons, location pins) to accommodate fast visual processing, and form fields are large and keyboard-friendly for rapid data entry.

4.2. Wireframes & Mockups

Based on the finalized Figma prototypes, the web application consists of four primary operational screens:

1. Secure Authentication Portal (Login Screen)

- *Layout:* A modern split-screen design. The left panel reinforces the system's mission ("Every second. Every signal. One response.") and displays live system health metrics (e.g., < 1 sec prediction response, 99.9% uptime). The right panel features a clean, secure login form for authorized personnel, ensuring role-based access control.

2. Incident Desk (Report & Predict View)

- *Layout:* The core operational dashboard for dispatchers. The left section contains an intuitive "Incident details" form for rapid input of location coordinates (Latitude/Longitude) and scene conditions (Weather, Lighting, Road type, Vehicle).
- *AI Integration:* The right section features a dynamic "AI Assessment" card. Upon prediction, this card visually shifts (e.g., turning solid Red for a "Serious" prediction with an 87% confidence score) and displays the algorithmic response time (e.g., 0.42 sec). It also tracks the automated hospital notification status in real-time.

3. Hospital Network (Live Alert Feed)

- *Layout:* Designed for hospital emergency rooms and ambulance hubs. The left pane displays a prioritized, real-time "Emergency feed" of incoming casualties sorted by severity.
- *Map Integration:* The right pane renders a geographic map showing the exact incident location, the estimated time of arrival (ETA), casualty counts, and a clear "Alert Acknowledged" call-to-action button for medical staff to confirm readiness.

4. Traffic Intelligence (Analytics Dashboard)

- *Layout:* A comprehensive analytical interface for traffic authorities. The top row features critical KPI cards (Total Collisions, Fatal Incidents, Serious Incidents, and Avg Response Time).

- *Visualizations:* The center displays a large interactive "Accident hotspot density" map (heatmap) to identify high-risk zones across the region. A severity distribution donut chart complements the data, allowing officials to make data-driven infrastructure decisions.

Prototype: Emergency Response System – Figma Make

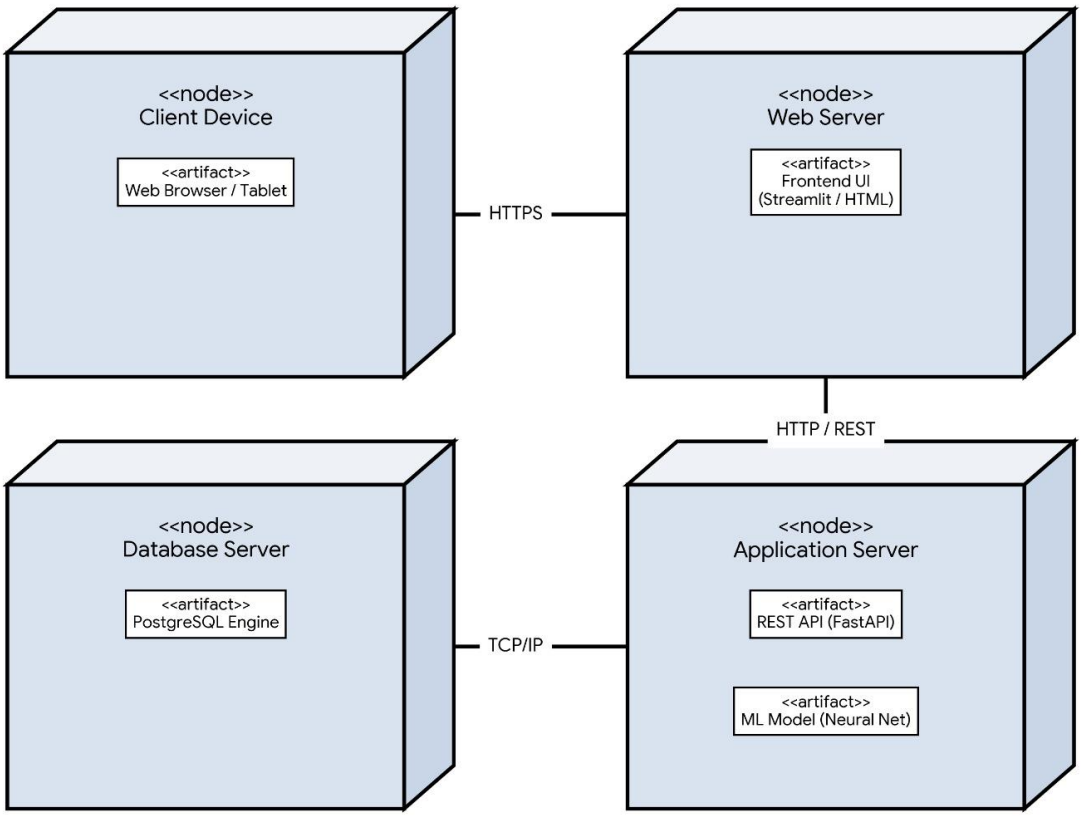
5. System Deployment & Integration

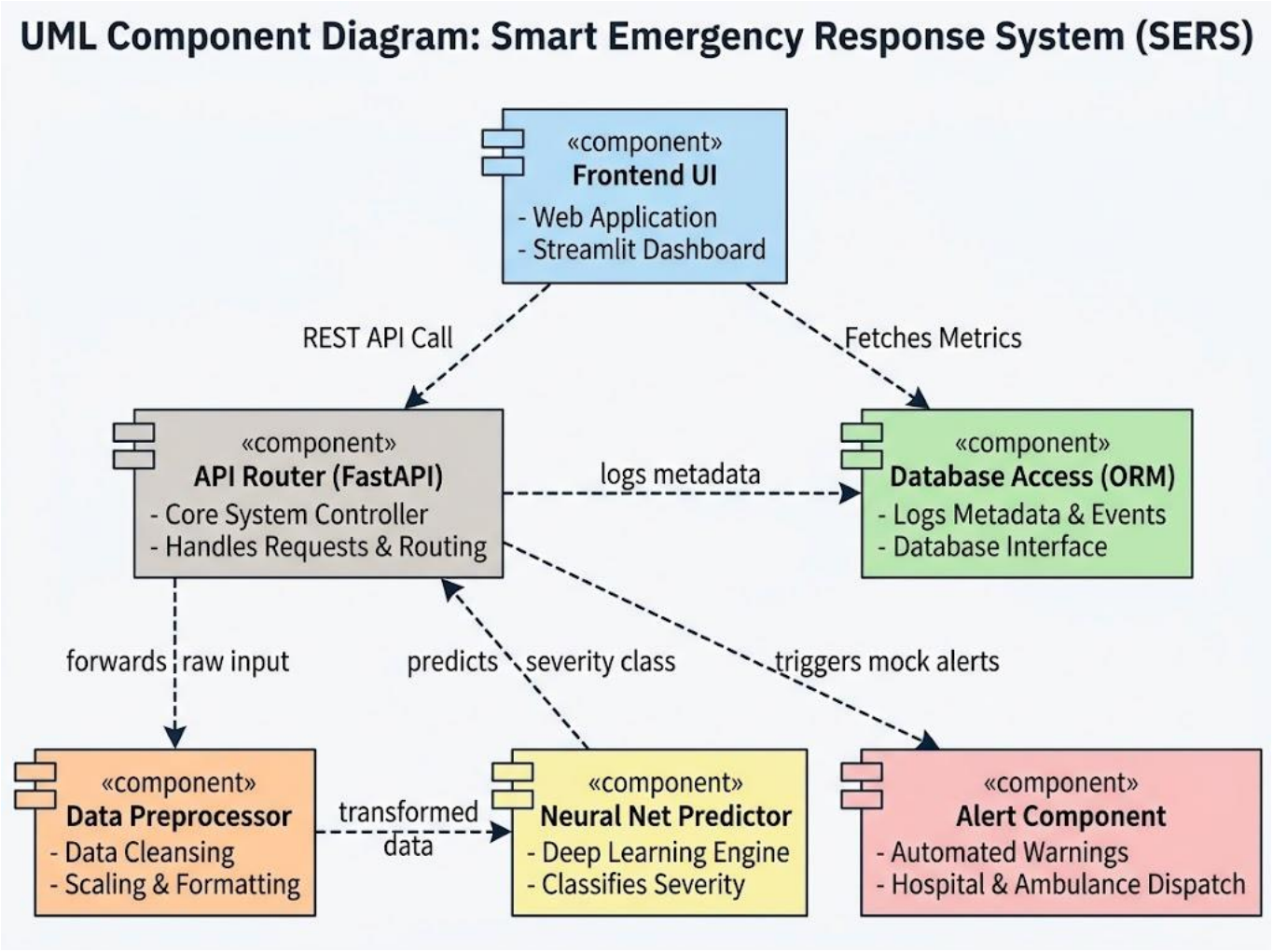
Technology Stack

The Smart Emergency Response System (SERS) is built using a modern, scalable, and modular technology stack designed to support real-time Machine Learning inferences and high-concurrency API requests.

- **Frontend (User Interface & Dashboard):**
  - *Streamlit* (or standard HTML5/CSS3/JavaScript) for developing the interactive analytics dashboard, heatmaps, and the emergency dispatcher input forms.
- **Backend (API & Business Logic):**
  - *FastAPI (Python)* for building a lightning-fast, asynchronous RESTful API to handle client requests, trigger alerts, and serve the ML model.
- **Machine Learning & Data Processing:**
  - *TensorFlow/Keras or PyTorch* for building and training the Deep Learning (Neural Network) model.
  - *Pandas and NumPy* for data manipulation and preprocessing of the STATS19 dataset.
- **Database Management:**
  - *PostgreSQL* (or MongoDB) for secure, scalable logging of accident reports, predictions, and user credentials.
- **Deployment & DevOps:**
  - *Docker* for containerizing the application environments.
  - *GitHub* for version control and CI/CD pipelines.
  - *AWS, Render, or Heroku* for cloud hosting the backend, frontend, and database instances.

UML Deployment Diagram: Smart Emergency Response System (SERS)





6. Additional Deliverables

6.1. API Documentation (FastAPI Endpoints)

The backend of the Smart Emergency Response System (SERS) is powered by FastAPI, which automatically generates interactive OpenAPI (Swagger) documentation. Below are the core API endpoints developed for the system:

- **1. POST /api/v1/predict-severity**
  - *Description:* Accepts real-time accident data from the dispatcher frontend, processes it through the Deep Learning (Neural Network) model, and returns the predicted severity.
  - *Payload (JSON):* { "latitude": float, "longitude": float, "weather_condition": int, "light_condition": int, "vehicle_type": int, "road_type": int }
  - *Response (JSON):* { "incident_id": string, "predicted_severity": "Serious", "confidence_score": 0.87, "inference_time_sec": 0.42 }
- **2. POST /api/v1/dispatch-alert**
  - *Description:* Triggered internally if the severity is predicted as "Fatal" or "Serious". It queries the database for the nearest medical facility and sends a simulated webhook/email alert.
  - *Payload (JSON):* { "incident_id": string, "severity": string, "location_coords": list }
- **3. GET /api/v1/dashboard-metrics**
  - *Description:* Fetches aggregated historical data (from the STATS19 dataset) and live incident logs to populate the Traffic Intelligence dashboard.
  - *Query Parameters:* ?region=string&time_range=string

6.2. Testing & Validation

To ensure a robust and reliable emergency platform, the system will undergo three levels of testing:

- **1. Unit Testing:**
  - Using the pytest framework in Python to test isolated components. This includes testing the data preprocessing functions to ensure missing values are handled correctly, and
  -
  - testing the Neural Network's inference function to ensure it outputs the correct format without crashing.
- **2. Integration Testing:**
  - Using tools like Postman and FastAPI's TestClient to verify that the components communicate seamlessly. This ensures that when the API receives data, it successfully triggers the ML model, logs the result in PostgreSQL, and initiates the alert component.
- **3. User Acceptance Testing (UAT):**
  - Conducting simulated emergency scenarios with team members acting as "Dispatchers" and "Hospital Managers" to validate that the UI is intuitive, the prediction is instantaneous, and the dashboard metrics update correctly.

6.3. Deployment Strategy

The system is designed for high availability and scalability using modern DevOps practices:

- **Hosting Environment:**
  - *Backend & ML Model:* Hosted on cloud platforms such as AWS (EC2/Elastic Beanstalk) or Render, providing sufficient compute resources for the Neural Network inference.

- *Frontend:* Deployed via Vercel or Streamlit Community Cloud for fast, CDN-backed delivery.
- *Database:* A managed PostgreSQL instance (e.g., AWS RDS or Supabase) to ensure secure and persistent data storage.

- **Deployment Pipelines (CI/CD):**
  - Utilizing **GitHub Actions** to automate the deployment process. Every code push to the main branch will automatically trigger unit tests; if successful, the new build is deployed to the production server.
- **Scaling Considerations:**
  - The backend application will be containerized using **Docker**, ensuring consistency across environments. FastAPI's asynchronous capabilities (async/await) combined with a production server like Uvicorn will allow the system to handle multiple concurrent emergency reports without bottlenecking.

5. Implementation (Source Code & Execution)

1. Source Code

• Structured & Well-Commented Code – Clean, maintainable, and properly documented code following best practices.

• Coding Standards & Naming Conventions – Consistent formatting, meaningful variable names, and adherence to industry standards.

• Modular Code & Reusability – Organized into reusable components, functions, and classes.

• Security & Error Handling – Secure coding practices, validation checks, and proper exception handling.

2. Version Control & Collaboration

• Version Control Repository – Hosted on GitHub, GitLab, or Bitbucket with a public/private repository link.

• Branching Strategy – Clear workflow (e.g., GitFlow, Feature Branching) for managing code updates.

• Commit History & Documentation – Meaningful commit messages and detailed pull request descriptions.

• CI/CD Integration (if applicable) – Automated builds, testing, and deployment pipelines.

3. Deployment & Execution

- **README File – Includes:**
- **Installation steps**
- **System requirements (hardware/software dependencies)**

- **Configuration instructions**
- **Execution guide (running the project locally or accessing a deployed version)**
- **API documentation (if applicable)**
- **Executable Files & Deployment Link –**
- **Compiled software or packaged application (e.g., .exe, .jar, .apk).**
- **Deployed web/mobile app**

---

6. Testing & Quality Assurance

- **Test Cases & Test Plan** – Document detailing test scenarios and expected outcomes.
- **Automated Testing (if applicable)** – Any automated test scripts used.
- **Bug Reports** – Issues identified and resolutions.

---

7. Final Presentation & Reports

- **User Manual** – Instructions for end users.
- **Technical Documentation** – System architecture, database schema, API documentation.
- **Project Presentation (PPT/PDF)** – Summary of the project, challenges, solutions, and outcomes.
- **Video Demonstration (Optional)** – Short demo showcasing the project's functionality.
- **Project Infographics: a single image represents main ideas in the project You can use the template below**

Info Graphic template (to be used by all projects)

You are an award-winning infographic designer, information architect, AI visual storyteller, and senior presentation designer.

Your job is to transform an uploaded project document into ONE premium presentation-quality roadmap infographic.

The final result must look like it was designed by a senior presentation designer at Google, Microsoft, AWS, NVIDIA, or OpenAI.

==========
STEP 0 — DEPI INFORMATION (MANDATORY)
==========

Before generating the infographic, ask the user:

"Please enter your DEPI Track Name."

Examples:
* AI & Data Science
* Data Engineering
* Full Stack .NET
* Full Stack Python
* Front-End Development
* DevOps
* Cloud Computing
* Cybersecurity
* Embedded Systems
* Mobile Development

Never assume the track name.

After the user provides it, display the following presentation header:

DEPI
Digital Egypt Pioneers Initiative

Track:
<User Track Name>

If an official DEPI logo is available, place it in the top-left corner.

Otherwise create a clean professional DEPI wordmark.

The branding should occupy approximately 8% of the slide height.

Keep it elegant and minimal.

==========
PRIMARY OBJECTIVE
==========

Read the ENTIRE uploaded document.
Understand it completely before designing anything.
Create ONE SINGLE premium presentation slide that summarizes the entire project.
The slide should communicate the whole project within 30 seconds.
The viewer should not need to read the original document.
Everything must fit naturally inside ONE presentation slide.

==========
CANVAS
==========
Presentation Slide
Landscape
Aspect Ratio:16:9
Resolution: Preferred: 2560 × 1440
Minimum: 1920 × 1080
Never generate:
* Poster
* Multi-page design
* Long infographic
* Vertical layout
* Scrolling layout

Everything must fit naturally inside ONE slide.

==========
STEP 1 — UNDERSTAND THE DOCUMENT
==========

Read the entire document.
Extract ONLY information explicitly found inside the document.

Do NOT invent information.
Do NOT assume technologies.

Do NOT fabricate architecture.
Identify:

* Project Title
* One-sentence Summary
* Business Problem
* Proposed Solution
* Target Users
* Core Features
* System Components
* Technical Architecture
* AI Components
* Technology Stack
* External Services
* APIs
* Database
* Deployment Strategy
* Development Phases
* Deliverables
* Future Enhancements

If a section doesn't exist,
omit it completely.

==========

STEP 2 — ORGANIZE CONTENT

==========

Transform everything into a left-to-right roadmap.
Suggested order:
Project Overview
Business Problem
Solution
Core Features
Architecture
AI Components
Technology Stack
Development Timeline
Deployment
Deliverables
Future Improvements
Use keywords instead of paragraphs.
Maximum 2–4 bullets per section.

==========

VISUAL STYLE

==========

Create a premium engineering presentation.
Style:
* Executive
* Minimal
* Elegant
* Engineering
* Professional
* Modern
* Corporate
* Presentation-ready

Inspired by:
Google
Microsoft
AWS
OpenAI
NVIDIA
Apple Keynote
Coursera

==========

COLOR PALETTE

==========

Primary:
Deep Blue

Secondary:
Purple
Accent:
Cyan
Highlight:
Orange
Success:
Green
Background:
White or Light Gray
Use:

* Soft gradients
* Rounded cards
* Thin connectors
* Premium shadows
* Glassmorphism (light)
* Clean spacing

==========
LAYOUT
==========

Horizontal roadmap.
Flow:
LEFT
RIGHT

Connect sections using:
* arrows
* workflow lines
* timeline connectors
* curved paths

Everything should align on a clean presentation grid.

Avoid clutter.
Avoid empty space.

==========
ROADMAP SECTIONS
==========

Each roadmap block must include:

* icon
* short title
* 2–4 keywords

No paragraphs.
No long text.

==========
SYSTEM ARCHITECTURE
==========

If the project contains AI,
draw an architecture similar to:
User
Frontend
Backend API
LangGraph Orchestrator (or equivalent from the document)
AI Models / LLM
Shared Services / Tools
Memory / Cache

Database
External APIs
Dashboard / Results
Replace the architecture with the actual one described in the document.
Never invent missing layers.
==========
TECH STACK
==========
Display technologies as modern badges.
Examples:
Python
FastAPI
Flask
Django
React
Angular
Vue
Node.js
LangGraph
LangChain
CrewAI
AutoGen
OpenAI
Gemini
Claude
Groq
Llama
Docker
Kubernetes
PostgreSQL
MongoDB
Redis
SQLAlchemy
Plotly
Streamlit
Pandas
NumPy
scikit-learn
Prophet

PyTorch
TensorFlow
GitHub

ONLY display technologies explicitly mentioned.

==========
AI COMPONENTS
==========
If AI exists,
display AI modules separately.
Examples:
LLM
Multi-Agent System
Forecasting
RAG
Memory
Prompt Engineering
Analytics
Customer Segmentation
Prediction
Visualization
Show relationships visually.
==========
DEVELOPMENT TIMELINE
==========

Create a compact timeline.
Only include phases found inside the document.
Example:
Planning
Design
Implementation
Testing
Deployment
Maintenance

==========
PROJECT OUTCOMES
==========

At the bottom create:
Project Outcomes
Display only deliverables found inside the document.
Examples:

- Working System
- Dashboard
- Source Code
- Documentation

REST API
AI Model
Deployment
Web Application

==========

TYPOGRAPHY

==========

Modern presentation typography.
Large title.
Strong hierarchy.
Balanced spacing.
Excellent readability.
High contrast.

==========

ICONS

Automatically choose premium outline icons.
Examples:
AI
Brain
Database
Analytics
Dashboard
Workflow
Security
Cloud
Users
Server
Backend
Frontend
API
Deployment
Monitoring
Automation

==========

FINAL QUALITY REQUIREMENTS

==========

The output MUST look like a slide from:
Google I/O
Microsoft Build
AWS re:Invent
NVIDIA GTC

OpenAI DevDay
NOT a poster.
NOT a report.

NOT a research paper.
NOT multiple pages.
Prioritize:

* Executive quality
* Minimal text
* Beautiful hierarchy
* Premium spacing
* Modern UI
* High information density
* Large icons
* Presentation aesthetics

The infographic must be immediately usable inside:
PowerPoint
Google Slides
Keynote
without requiring redesign.
The final slide should be understandable in under 30 seconds while maintaining a premium conference-quality appearance.